

AYUDANTIA 1

Computación gráfica y aplicaciones: Introducción a la computacion gráfica

Profesor:
Alfonso Ehijo

Ayudante:
Alan E. Molina H.
alan.molina@pregrado.uoh.cl

COM3201-1

TABLA DE CONTENIDO

3	Introduccion
5	Elementos basicos
6	Tipo de datp
7	Operaciones básicas
13	Diagrama de flujo
14	Ejercicios diagrama de flujo
16	Python cientifico
20	Ejercicios programacion

Introducción

¿En qué áreas puedo aplicar python ?



Introducción

Sintaxis

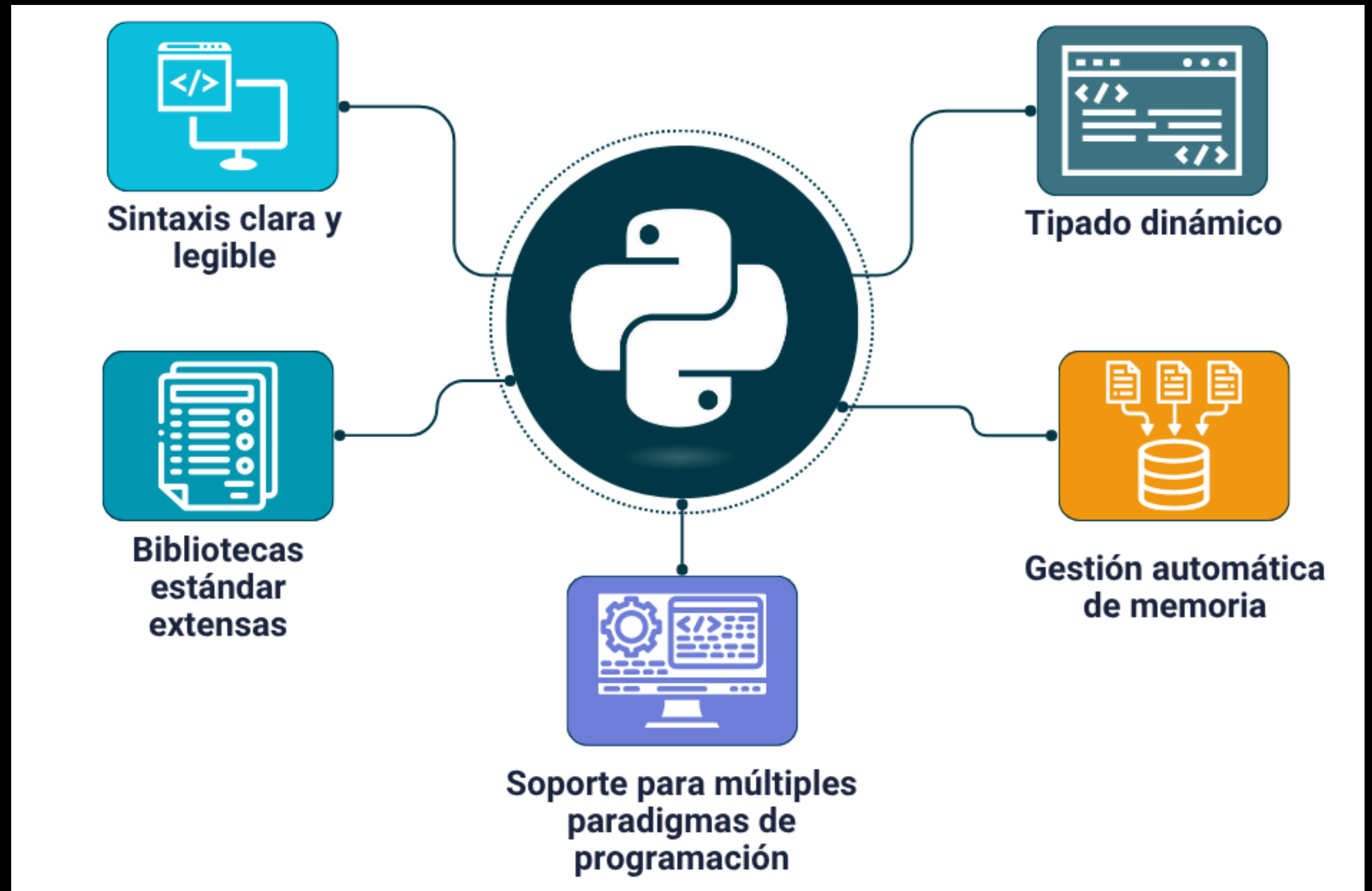
- Identificadores (nombre para variable, funciones, etc)
- Letras, underscore (_) y números
- No usar palabras reservadas. Como: false, continue.
- Nombre de clases comienzan con mayúscula (convención)

Líneas e indentación:

- Bloques se delimitan con indentación de líneas
- No se utilizan llaves {} ni punto y coma ;
- Comentarios con #



Características



Elementos básicos de python

SINTAXIS BÁSICA DE PYTHON

Python se usa en machine learning, web, seguridad informática, data science, cloud computing y tiene una de las sintaxis más sencillas.



```
1 #!/usr/bin/python3
2 import sys
3 def holamundo():
4     for i in range(1, 10):
5         nombre = 'EDteam'
6         print("Hola Mundo " + nombre)
7 if __name__ == "__main__":
```

Comentarios con **#** al inicio

Importación de módulos

Las funciones se definen con **def** y el nombre de la función.

Los ciclos **for** pueden indicar un rango o una estructura de datos para recorrer.

A_xM

TIPOS DE DATOS EN PYTHON



NUMERICOS

INT()

Números enteros positivos o negativos



```
Edad = 25 # Un numero entero
```

FLOAT()

Números con decimales

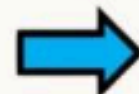


```
Edad = 25,78 # Un numero decimal
```

CADENAS DE TEXTO

STR()

Cadena o texto



```
Nombre = "Jose" # Una cadena de texto
```

BOOLEANOS

BOOL()

Valores de verdadero o falso



```
es_cliente = True # Verdadero o Falso
```

NINGUNO O NULL

NONE TYPE()

Representa la ausencia de valor



```
resultado = None # Sin valor asignado
```

Tipos de datos

Operaciones básicas

Operador	Nombre	Ejemplo
	Suma	$x + y = 13$
	Resta	$x - y = 7$
	Multiplicación	$x * y = 30$
	División	$x / y = 3.333$
	Módulo	$x \% y = 1$
	Exponente	$x ** y = 1000$
	Cociente	3

```
x = 10; y = 3
print("Operadores aritméticos")
print("x+y =", x+y)    #13
print("x-y =", x-y)    #7
print("x*y =", x*y)    #30
print("x/y =", x/y)    #3.3333333333333335
print("x%y =", x%y)    #1
print("x**y =", x**y)  #1000
print("x//y =", x//y)  #3
```

Operador	Descripción	Ejemplo
==	¿son iguales a y b?	>>> False
!=	¿son distintos a y b?	>>> True
<	¿es a menor que b?	>>> False
>	¿es a mayor que b?	>>> True
<=	¿es a menor o igual que b?	>>> True
>=	¿es a mayor o igual que b?	>>> True

Asignación y operadores lógicos

Operador	Nombre
and	Devuelve True si ambos elementos son True
or	Devuelve True si al menos un elemento es True
not	Devuelve el contrario, True si es Falso y viceversa

Operador	Ejemplo	Equivalente
=	$x=7$	$x=7$
+=	$x+=2$	$x=x+2 = 7$
-=	$x-=2$	$x=x-2 = 5$
=	$x=2$	$x=x*2 = 14$
/=	$x/=2$	$x=x/2 = 3.5$
%=	$x\%=2$	$x=x\%2 = 1$
//=	$x//=2$	$x=x//2 = 3$
=	$x=2$	$x=x**2 = 49$
&=	$x\&=2$	$x=x\&2 = 2$
=	$x =2$	$x=x 2 = 7$
^=	$x^=2$	$x=x^2 = 5$
>>=	$x>>=2$	$x=x>>2 = 1$

Operadores

- Numéricos: +, -, *, /, **, %, //
- Asignación: =, +=, -=, *=, /=
- Comparación: ==, !=, <, >, <=, >=
- A nivel de bits: &, |, ^, ~, >>, <<
- Lógicos: **and**, **or**, **not**
- Membrecía: **in**, **not in**
- Identidad: **is**, **is not**

AND		
a	b	out
0	0	0
0	1	0
1	0	0
1	1	1

OR		
a	b	out
0	0	0
0	1	1
1	0	1
1	1	1

XOR		
a	b	out
0	0	0
0	1	1
1	0	1
1	1	0

NOT		
in	out	
0	1	
1	0	

Estructuras de control

If, Elif y Else

```
x = 5
if x == 5:
    print("Es 5")
elif x == 6:
    print("Es 6")
elif x == 7:
    print("Es 7")
else:
    print("Es otro")
```

for

```
for i in range(0, 5):
    print(i)
```

```
# Salida:
# 0
# 1
# 2
# 3
# 4
```

```
for i in "Python":
    print(i)
```

```
# Salida:
# P
# y
# t
# h
# o
```

range

```
for i in (0, 1, 2, 3, 4, 5):
    print(i) #0, 1, 2, 3, 4, 5
```

```
for i in range(6):
    print(i) #0, 1, 2, 3, 4, 5
```

```
for i in range(5, 20, 2):
    print(i) #5,7,9,11,13,15,17,19
```


Estructuras de control

While

```
x = 5
while x > 0:
    x -=1
    print(x)

# Salida: 4,3,2,1,0
```

While true

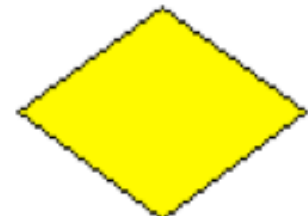
```
# No ejecutes esto, en serio
while True:
    print("Bucle infinito")
```

While Anidado

```
# Permutación a generar
i = 0
j = 0
while i < 3:
    while j < 3:
        print(i,j)
        j += 1
    i += 1
    j = 0
```

Herramientas de programación científica

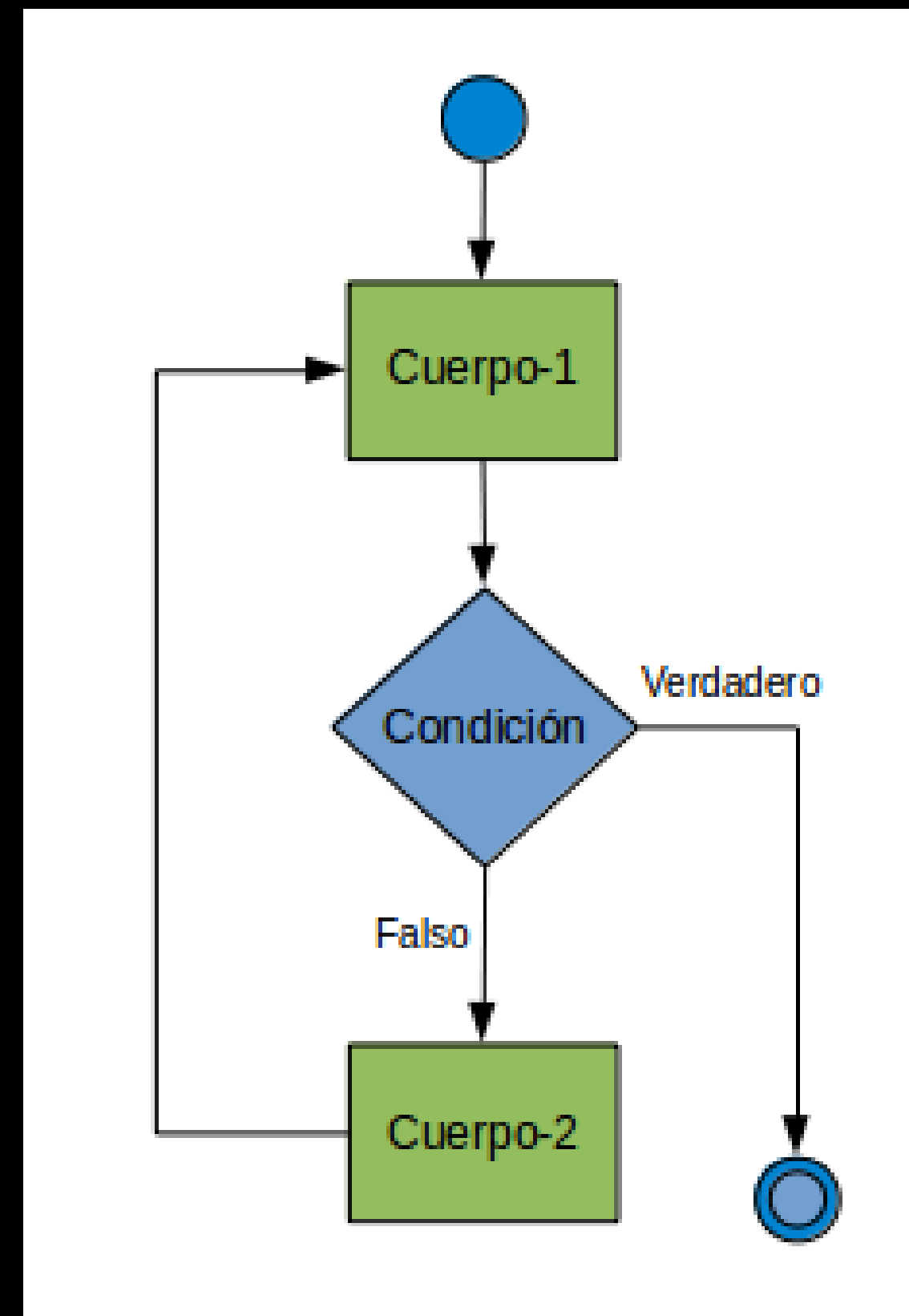
- Diagrama de flujo:
Elementos básicos

	Inicio/Final Se utiliza para indicar el inicio y el final de un diagrama; de Inicio sólo puede salir una línea de flujo y al final sólo debe llegar una línea		Decisión Indica la comparación de dos datos y dependiendo del resultado lógico (falso o verdadero) se toma la decisión de seguir un camino del diagrama u otro
	Entrada/Salida Entrada/Salida de datos por cualquier dispositivo (scanner, lector de código de barras, micrófono, parlantes, etc.)		Impresora/Documento. Indica la presentación de uno o varios resultados en forma impresa
	Entrada por teclado. Entrada de datos por teclado. Indica que el computador debe esperar a que el usuario teclee un dato que se guardará en una variable o constante		Pantalla Instrucción de presentación de mensajes o resultados en pantalla
	Acción/Proceso Indica una acción o instrucción general que debe realizarse (operaciones aritméticas, asignaciones, etc.)		Conector Interno Indica el enlace de dos partes de un diagrama dentro de la misma página

Python científico

Estructuras de control de flujo

- Condicionales:
if, elif, else
- Ciclos:
for, while, break, continue
- Funciones de python:
zip, range, enumerate



Herramientas de programación científica

- **Diagramas de flujo (ejercicios)**
 - i. Imprimir “hola mundo”
 - ii. Calcular el área de un círculo. Considerar $\pi = 3.14$
 - iii. Obtener el mayor entre 2 números
 - iv. Imprimir solo si el número es mayor que cero

Bonus: sumar todos los números pares de un vector de tamaño n

Python científico

Tipos de datos

- Números*: `int`, `float`, `long`, `complex`, `bool`
- Secuencias: `str`*, `list`, `tuple`*, `dict`, `set`*
- Funciones
- Clases

En Python,

¡Todo tipo de datos es un objeto!

*Immutable

Categoría	Ejemplos principales	Uso típico
Matemáticas y números	abs, divmod, pow, round, sum, min, max	Cálculos básicos
Colecciones y secuencias	len, sorted, reversed, enumerate, zip	Manejo de listas/tuplas
Tipos y objetos	type, isinstance, isinstance, property	POO y validación
Entrada/Salida	print, input, open, help	Interacción con usuario
Ejecución y depuración	eval, exec, compile, breakpoint	Código dinámico
Introspección	dir, globals, locals, vars, id, hash, repr	Inspección del entorno
Especiales	object, memoryview, import , chr, ord	Funciones base

Funciones típicas en Python

Python científico

Funciones estándares

Built-in Functions				
<code>abs()</code>	<code>divmod()</code>	<code>input()</code>	<code>open()</code>	<code>staticmethod()</code>
<code>all()</code>	<code>enumerate()</code>	<code>int()</code>	<code>ord()</code>	<code>str()</code>
<code>any()</code>	<code>eval()</code>	<code>isinstance()</code>	<code>pow()</code>	<code>sum()</code>
<code>basestring()</code>	<code>execfile()</code>	<code>issubclass()</code>	<code>print()</code>	<code>super()</code>
<code>bin()</code>	<code>file()</code>	<code>iter()</code>	<code>property()</code>	<code>tuple()</code>
<code>bool()</code>	<code>filter()</code>	<code>len()</code>	<code>range()</code>	<code>type()</code>
<code>bytearray()</code>	<code>float()</code>	<code>list()</code>	<code>raw_input()</code>	<code>unichr()</code>
<code>callable()</code>	<code>format()</code>	<code>locals()</code>	<code>reduce()</code>	<code>unicode()</code>
<code>chr()</code>	<code>frozenset()</code>	<code>long()</code>	<code>reload()</code>	<code>vars()</code>
<code>classmethod()</code>	<code>getattr()</code>	<code>map()</code>	<code>repr()</code>	<code>xrange()</code>
<code>cmp()</code>	<code>globals()</code>	<code>max()</code>	<code>reversed()</code>	<code>zip()</code>
<code>compile()</code>	<code>hasattr()</code>	<code>memoryview()</code>	<code>round()</code>	<code>__import__()</code>
<code>complex()</code>	<code>hash()</code>	<code>min()</code>	<code>set()</code>	
<code>delattr()</code>	<code>help()</code>	<code>next()</code>	<code>setattr()</code>	
<code>dict()</code>	<code>hex()</code>	<code>object()</code>	<code>slice()</code>	
<code>dir()</code>	<code>id()</code>	<code>oct()</code>	<code>sorted()</code>	

Funciones

- Estructura general:

```
def nombre (parametro):  
    #parametro opcional  
    '''  
    documentacion  
    '''  
    #cuerpo de la funcion  
    expresion = not parametro  
    return expresion #(opcional)
```

```
nombre(True)
```

```
False
```

- Función

- Llamado

- Salida

- Función

- Llamado

- Salida

Funciones lambda:

- Estructura general:

```
f = lambda x,y:x*y
```

```
print(f(2,3))
```

```
6
```


Ejercicios propuestos

- i. Imprimir “hola mundo”
- ii. Calcular el área de un círculo. Considerar $\pi = 3.14$
- iii. Obtener el mayor entre 2 números
- iv. Imprimir solo si el número es mayor que cero
- v. Calcular factorial

Bonus: sumar todos los números pares de un vector de tamaño n